

Final Project Report: Interactive Pick-and-Place System Based on LeRobot

Team Number: Team 20

Team Members:

Sida Zhou (225010296) – Policy training & Robot control & Calibration (Research, Coding)

Huanzhang Hu (225040233) – Hardware Setup & Data Collection (Testing)

Siquan Wu (225040259) – Hardware Setup & Data Collection (Testing)

Hanlin Zhang (225085007)–Policy training & Deployment & Documentation (Research, Coding)

Haonan Zhang (225040289)–Policy training & Deployment & Documentation (Research, Coding)

1. Project Overview

1.1 Goal

This project builds a tabletop interactive pick-and-place system on a real LeRobot SO-101 robotic arm. The robot receives natural-language user commands, converts them into structured task descriptions, selects the appropriate control policy or scripted action, and completes grasping, transport, and placement on real objects such as cubes, boxes, and cups.

1.2 Innovation and Intelligence

The system separates high-level semantic reasoning from continuous motor control. A Qwen3.5-2B language model parses user intent into a fixed JSON-style schema. The learned policy suite includes Action Chunk with Transformer(ACT), Diffusion Policy, and SmolVLA, allowing the team to compare classic imitation learning, diffusion-based action generation, and VLA-style generalization.

- **Language control:** Natural-language instructions such as "place the cube in the box" are parsed into structured tasks and routed to policy execution.

- **Policy comparison:** ACT, Diffusion Policy, and SmolVLA were trained and evaluated, with SmolVLA used as the strongest policy for loop-task behavior.

1.3 Hardware Setup

The physical platform is a LeRobot SO-101 arm assembled, calibrated, and used for real-robot demonstrations. The workspace is a tabletop environment observed through RGB cameras, with robot joint states, gripper state, and short observation history used as policy inputs. Qualcomm RB8 edge deployment was not used in the current implementation; the main deployment focus was the host-computer LeRobot pipeline and real SO-101 execution.

2. Design and Implementation

Task Planning (The "Brain"): Our system is designed as an interactive language-conditioned robotic manipulation pipeline based on the LeRobot SO-101 robotic arm. The goal is not only to let the robot execute predefined motions, but also to make it respond to natural, flexible, and sometimes imprecise human instructions. The complete system connects three major parts: high-level language understanding, robot skill execution, and learned policy deployment.

In the updated implementation, Qwen is introduced as the high-level language understanding and task-planning module. When the user gives an instruction, Qwen first analyzes the command and determines its intent. The command may be a simple movement instruction, a rotation command, a fuzzy adjustment command, a reset command, or a higher-level object manipulation task. For example, instructions such as “move a little to the left,” “rotate slightly,” or “place the cube into the box” are interpreted by Qwen and converted into a more structured task representation.

For fuzzy instructions, Qwen helps convert vague language into bounded and executable parameters. Instead of requiring the user to provide exact distances or angles, the system can understand expressions such as “a little,” “slightly,” “more,” or “a bit” and map them to safe movement distances or rotation angles. This makes the interaction more natural and user-friendly while keeping the robot motion within a safe range.

After the command is interpreted, the system selects the appropriate execution path. For low-level deterministic actions, such as moving, rotating, opening or closing the gripper, and resetting the arm, the command is sent to scripted LeRobot skills. For higher-level manipulation tasks, especially pick-and-place behaviors, the system uses learned policies. In particular, SmolVLA is used to map language-conditioned visual observations and robot states into continuous robot actions. In this design, Qwen works as the high-level instruction parser and task router, while SmolVLA works as the learned vision-language-action policy for manipulation.

This division of labor makes the system more modular and robust. Qwen handles semantic understanding and fuzzy instruction processing, while LeRobot skills and SmolVLA handle physical execution. As a result, the robot can respond to both simple control commands and more complex natural-language manipulation instructions in the real tabletop environment.

Robot Skills (The "Actions"): The task-planning module is responsible for understanding user instructions and deciding how the robot should respond. In our project, this part is mainly implemented through SmolVLA’s language-conditioned policy ability together with a lightweight command-processing layer for simple and fuzzy instructions. For manipulation tasks such as “put the cube into the box” or “pick up the object and place it there,” the natural-language instruction is directly provided to SmolVLA. SmolVLA uses its vision-language backbone to interpret the instruction together with visual observations and robot state information. Our system relies on SmolVLA itself to connect language, visual input, and action prediction. This allows the robot to generate actions based on both what the user says and what the robot observes in the workspace.

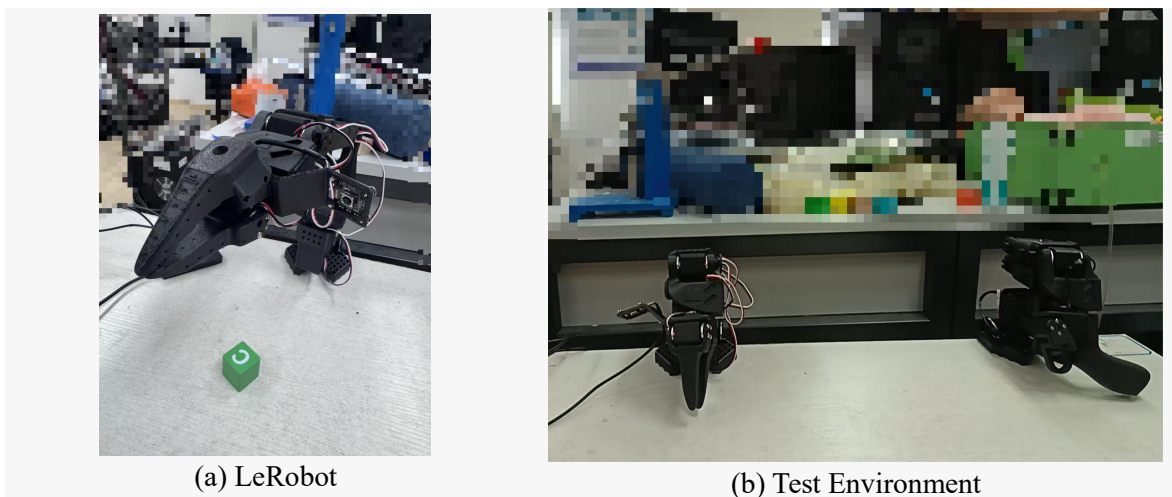
In addition to object-level manipulation commands, we also implemented fuzzy instruction handling. Many user commands are not given with exact numerical values. For example, a user may say “move a little to the left,” “rotate slightly,” or “move forward a bit.” These commands need to be converted into safe and executable robot parameters. Therefore, the command-processing layer maps vague language expressions into bounded movement distances or rotation angles. This makes the interaction more natural and reduces the need for the user to provide precise control values.

The task-planning module therefore supports two types of instruction handling. First, for high-level manipulation tasks, SmoVLA performs language-conditioned action generation. Second, for simple or fuzzy motion commands, the system converts the command into a corresponding LeRobot control primitive. This design keeps the system flexible: learned policies are used when perception and manipulation are required, while scripted skills are used when the command can be executed reliably with predefined motion primitives.

3. Experiments and Results

3.1 Test Environment

Experiments were conducted on a lab tabletop workspace using the real SO-101 arm as shown below. The environment included simple objects and containers such as a cube, a box, a cup, and tape-like cylindrical objects. The robot executed both single pick-and-place trials and interactive loop demonstrations triggered by natural-language commands.



3.2 Policy Training Tasks

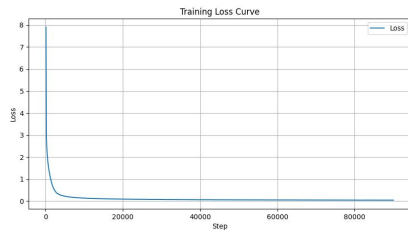
A major part of our project was real-robot data collection and policy training. We collected 200 demonstration trajectories in total, including 80 standard-position trajectories and 120 generalized-position trajectories. These demonstrations were collected for two main tasks: pick-and-place and loop manipulation. The standard-position data were used to train baseline policies under relatively fixed object layouts, while the generalized-position data were added to test whether the policies could handle changes in object positions and task layouts.

For the ACT baseline, we trained two ACT policies on the 80 standard-position trajectories. These ACT models were mainly used for the pick-and-place and loop-task baselines. ACT showed relatively stable behavior when the object positions were close to the demonstrations, but its ability to generalize was limited. In particular, when we tried to train or evaluate a bad ACT baseline with the more generalized 200-trajectory dataset, the robot often produced unstable or random motions instead of meaningful manipulation. This shows that simply increasing data diversity does not necessarily make ACT robust in our setting, especially when the model cannot effectively handle the larger variation in object positions.

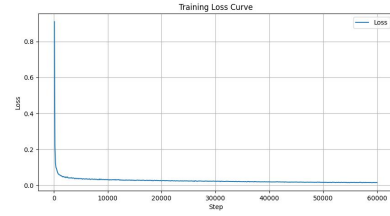
Diffusion Policy and SmolVLA were trained using the full 200-trajectory dataset, including both the 80 standard-position demonstrations and the 120 generalized-position demonstrations. Compared with ACT, Diffusion Policy was able to learn a more diverse action distribution from the dataset. One important observation is that the policy learned a bimodal behavior pattern in the loop task: in some rollouts, the robot first moved to grasp the tape on the right side, while in other rollouts, it first moved to grasp the tape on the left side. This behavior is consistent with our dataset, because both action modes appeared in the collected demonstrations. Therefore, Diffusion Policy successfully captured the multi-modal structure of the task.

However, although Diffusion Policy learned this bimodal distribution, its real-robot accuracy was still not satisfactory. The grasping position was often not precise enough, and the robot sometimes failed to reach the object reliably. In several trials, manual intervention was required: we had to place the tape closer to the robot or move it into a region that the arm could reach more easily. As a result, Diffusion Policy could demonstrate the learned task tendency, but it could not achieve reliable one-shot grasping in our real tabletop environment.

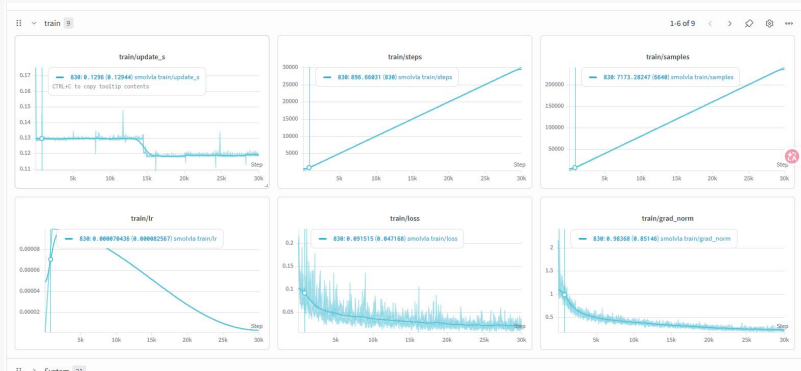
SmolVLA showed stronger generalization after training on the full 200-trajectory dataset. In the loop-task demonstrations, the tape positions at the two loop peaks were clearly different, but the trained SmolVLA model was still able to grasp the target accurately in both cases. This indicates that SmolVLA learned a more robust language-conditioned manipulation policy from the generalized dataset. Compared with the bad ACT result, where the robot tended to move randomly under generalized training data, SmolVLA produced more stable and meaningful actions when object positions changed.



(a) ACT pick-and-place training loss curve



(b) Diffusion Policy pick-and-place training loss curve



(c) SmolVLA pick-and-place training loss curve

3.3 Challenges and Solutions

The biggest challenge in this project was the gap between real-robot data collection, policy training, and stable execution on the LeRobot SO-101 arm. Since all demonstrations were collected by ourselves on the real SO-101 robot, the quality of the dataset directly affected the final policy performance. Small differences in object position, camera view, gripper alignment, joint calibration, or human demonstration style could lead to large differences during policy rollout.

During data collection, we first collected 80 standard-position trajectories. These demonstrations were useful for training baseline ACT policies because the object positions and task layout were relatively fixed. However, when the object positions changed, the ACT policy showed limited generalization. To improve this, we collected an additional 120 generalized-position trajectories, resulting in 200 real-robot demonstrations in total. These data were used to train and compare ACT, Diffusion Policy, and SmolVLA.

One major issue appeared when training and deploying policies on more generalized data. ACT worked relatively well on the standard-position dataset, but when we tried to train a `bad_ACT` baseline using the full 200-trajectory dataset, the robot sometimes produced unstable or meaningless motions. This showed that simply increasing data diversity did not automatically improve ACT performance in our real-robot setting.

Diffusion Policy learned more diverse behaviors from the dataset. In the loop task, it captured a bimodal distribution: in some trials, the robot first tried to grasp the tape on the right side, while in other trials it first tried to grasp the tape on the left side. This matched the demonstrations in our dataset, where both action modes appeared. However, its real-robot accuracy was still limited. The robot did not always grasp the tape precisely, and sometimes human intervention was needed to manually move the tape into a more reachable position. Therefore, Diffusion Policy learned the overall behavior pattern, but it was not reliable enough for accurate one-shot grasping.

SmolVLA achieved better generalization in our experiments. After training on the full 200 real-robot trajectories, the model showed more stable behavior in the loop task. In our demonstrations, the tape positions at two loop peaks were clearly different, but SmolVLA was still able to grasp the target accurately. Compared with `bad_ACT`, which tended to move randomly under generalized training data, SmolVLA produced more meaningful and robust actions when the object positions changed.

To address these challenges, we improved the system in several ways. First, we carefully calibrated the LeRobot SO-101 arm and kept the task within a safe and reachable tabletop workspace. Second, we collected both standard-position and generalized-position demonstrations to support different training settings. Third, we compared ACT, Diffusion Policy, and SmolVLA instead of relying on a single policy.

4. Photo and Video Documentation

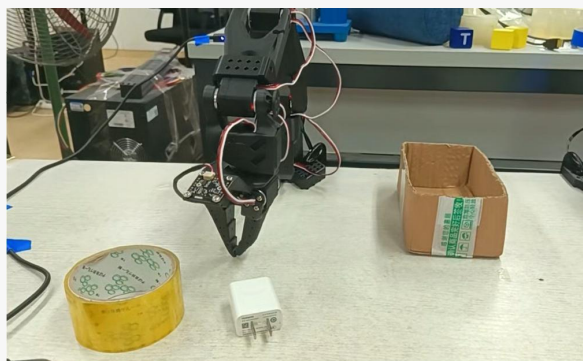
Limited by .doc and .pdf file, we can't upload the video documents. We just upload these videos and supplemental materials in CUHKSZ's OneDrive for reviewing:

https://cuhko365-my.sharepoint.com/:f:/g/personal/225085007_link_cuhk_edu_cn/IgC-S8iPNIKCTYIYEQTsU50dAfWFLRbdwzIciJkiGMugbpE?e=U5aiF3

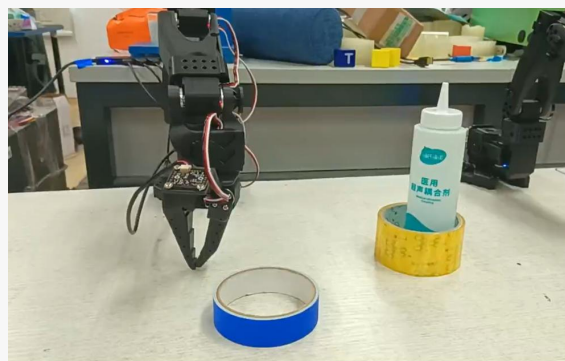
我的文件 > AIR5021-Final_Report_Material

名称	修改时间	修改者	文件大小	共享	活动
1. Demof for ACT、Diffusion、smolVLA	3 分钟前	Hanlin Zhang (SAI,225085007)	8 个项目	已共享	
2. Demo for LLM to analyze NLP order	3 分钟前	Hanlin Zhang (SAI,225085007)	8 个项目	已共享	
3. loss	3 分钟前	Hanlin Zhang (SAI,225085007)	8 个项目	已共享	

(a) OneDrive Link for Photo and Video Document



(b) Pick & place task



(c) Put tapes on the bottle

5. Conclusion

5.1 Final Achievement

In this project, we successfully built an interactive robotic manipulation system based on the real LeRobot SO-101 robotic arm. The system can receive user instructions, understand different types of commands, and execute corresponding robot behaviors in a tabletop environment. With Qwen as the high-level language understanding module, the robot can process natural-language commands, including simple movement instructions, rotation commands, fuzzy adjustment instructions, and object-level manipulation tasks

On the action side, we implemented several LeRobot-based robot skills, including movement, rotation, gripper control, grasping, pick-up, transport, placement, reset, and safe idle behaviors. These skills allow the robot to perform both direct interactive control and complete pick-and-place demonstrations.

We also collected our own real-robot demonstration dataset and trained three policy families: ACT, Diffusion Policy, and SmolVLA. The dataset contains 80 standard-position trajectories and 120 generalized-position trajectories, covering both pick-and-place and loop tasks. ACT served as a baseline on the standard-position dataset. Diffusion Policy learned multimodal behaviors, such as grasping either the left or right tape first in the loop task, but its real-robot precision was limited. SmolVLA showed the strongest generalization, successfully grasping tapes at clearly different positions in loop demonstrations.

5.2 Future Improvements

There are several directions for future improvement. First, the dataset can be further expanded. Although we collected 200 real-robot trajectories, more demonstrations with different object positions, object types, lighting conditions, and camera viewpoints would help improve policy robustness and reduce failure cases.

The accuracy of real-robot grasping can be improved. Diffusion Policy learned the overall bimodal behavior in the loop task, but it sometimes required manual intervention because the grasping position was not precise enough. Future work can improve calibration, workspace design, camera placement, and policy training settings to make the robot grasp more reliably without human adjustment. Then, the language interaction can be made more flexible. At the current stage, Qwen is mainly used for instruction understanding, command classification, and fuzzy instruction processing. In the future, the system could support more complex multi-step instructions, such as “pick up the tape, put it into the box, then return to the initial position,” and automatically decompose them into executable robot skills. The learned policies can be further compared and optimized. ACT, Diffusion Policy, and SmolVLA showed different strengths and weaknesses in our experiments. Future work can perform more quantitative evaluation, such as success rate, average completion time, grasping accuracy, number of manual interventions, and robustness under randomized layouts.

Finally, the system can be extended to more complex manipulation scenarios. For example, the robot could handle more object categories, cluttered environments, obstacle avoidance, and longer-horizon tasks. With more data and stronger policy training, the SO-101 platform could become a more reliable language-guided manipulation system for real-world tabletop tasks.

Qualcomm Device Usage (appendix, if applicable): None